

2022 삼성전자 (SR)

하계 인턴 프로젝트 발표

[웨어러블 로봇] IMU 기반 동작 인식 알고리즘 개발

Date

2022.08.17.수요일

Organization

삼성전자 (SR) Robot Center GEMS LAB

Contents

자기 소개

Project 소개 및 개요

진행 과정 & 결과

- Timeline
- 전체 과정
- 세부 진행 과정
- Result

배운 점, 보완점

실무 OJT

■ LEE JOOHYUN / 이주현



LEE JOOHYUN
이주현

삼성리서치(SR) 인턴
SW 개발
GEMS LAB
(Robot Center)

[관심 개발 분야]
Sensor Fusion
Robot Perception
SLAM

■ Education

- 용인한국외국어대학교부설고등학교 졸업
- 성균관대학교 기계공학부 재학 중 (2023.02 졸업예정)

■ Experience

- 2020.03 ~ 현재 : 교내자율주행자동차 동아리 HEVEN 활동 – PL (프로젝트 리더)
- 실제 환경과 드론의 Dynamics를 활용한 Simulation 드론 대회 제작 (대학 ICT 연구센터 창의 자율 과제) – 과기부장관상
- 요양병원 내 로봇 심부름 플랫폼 개발 (교내 S-HERO 공학 인재 양성사업) – 우수상
- 3D 라이다를 이용한 다중 객체 추적 알고리즘 개발 (학부연구생)
- Depth 카메라 활용 A-pillar 사각지대 시야 확보 (자동차 기술아이디어 대회) – 금상
- 스마트카 융합 설계 경진대회 – 디자인 부문 우수상

■ Skills

- Python ☒☒☒☒☒☒☒☐☐☐
- C++ ☒☒☒☒☒☐☐☐☐☐ 사내 SW 검정 Pro 취득
- Ubuntu ☒☒☒☒☒☒☐☐☐☐
- ROS ☒☒☒☒☒☒☐☐☐☐
- English ☒☒☒☒☒☒☒☐☐☐ TOEIC 965 / OPIc IH

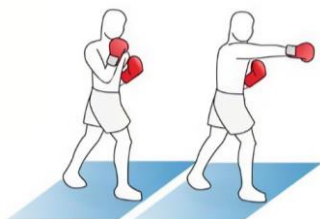
Project 소개 및 개요

■ Project 소개

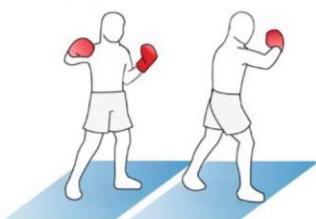
- 주제 : [웨어러블 로봇] IMU 기반 동작 인식 알고리즘 개발
 - 웨어러블 로봇의 IMU 데이터를 이용하여 사용자 동작 인식
 - Keywords : Action Recognition, HRI
- 세부 Task
 - 타겟 보드 (Nicla Sense ME-BHI260AP) 와의 통신, 센서 로그 데이터 수집 환경 구축
 - 사용자 동작 인식 관련 Home Fitness / Game / Boxing 동작 종류 선정
 - IMU 데이터 수집 및 전처리
 - 인식 알고리즘 모델 선정 및 개발
 - 웨어러블 로봇 로그 데이터 수집 및 모델 적용
 - 성능 검증
 - [optional] recognition on-the-fly
- 결과물 적용 대상 : 웨어러블 로봇 (from SR GEMS LAB)

■ Project 개요

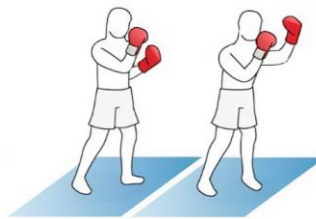
- 인식 동작 (6가지)
 - RS (Right Straight), RH (Right Hook), RU (Right Upper Cut)
LS (Left Straight), LH (Left Hook), LU(Left Upper Cut)



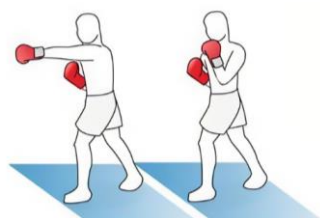
Right Straight



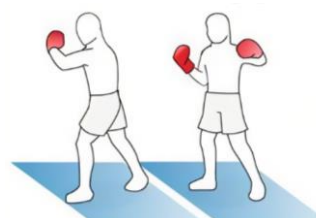
Right Hook



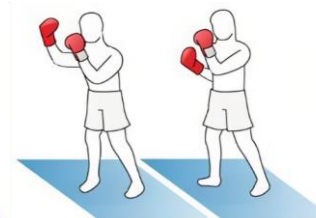
Right Upper Cut



Left Straight



Left Hook



Left Upper Cut

- 데이터 수집
 - 피실험자 5명, 동작 별 데이터 200개
- 분류 모델
 - SVM, ANN, CNN
 - SVM : Model 1 (100%), Model 2 (100%), Model 3 (77.78%)
 - CNN : Model 2 (61.11%), Model 3 (77.78%)
 - ANN : One Hand Triple Classifier (94.44%)

1주차

- 개발환경 구축
- 센서 보드 연결 (BLE 통신)
- 센서 데이터 로깅 코드 작성

3주차

- 센서 데이터 Pre-processing
- Feature Extraction
- Feature Selection
- 2진 분류 모델 구현 (SVM, ANN)

5주차

- 2중 3진 분류 모델 구현 (two hand) (SVM, CNN)
- Recognition on-the-fly (two hand)
- Noisy Data Generation
- 센서 데이터 Pre-processing 추가

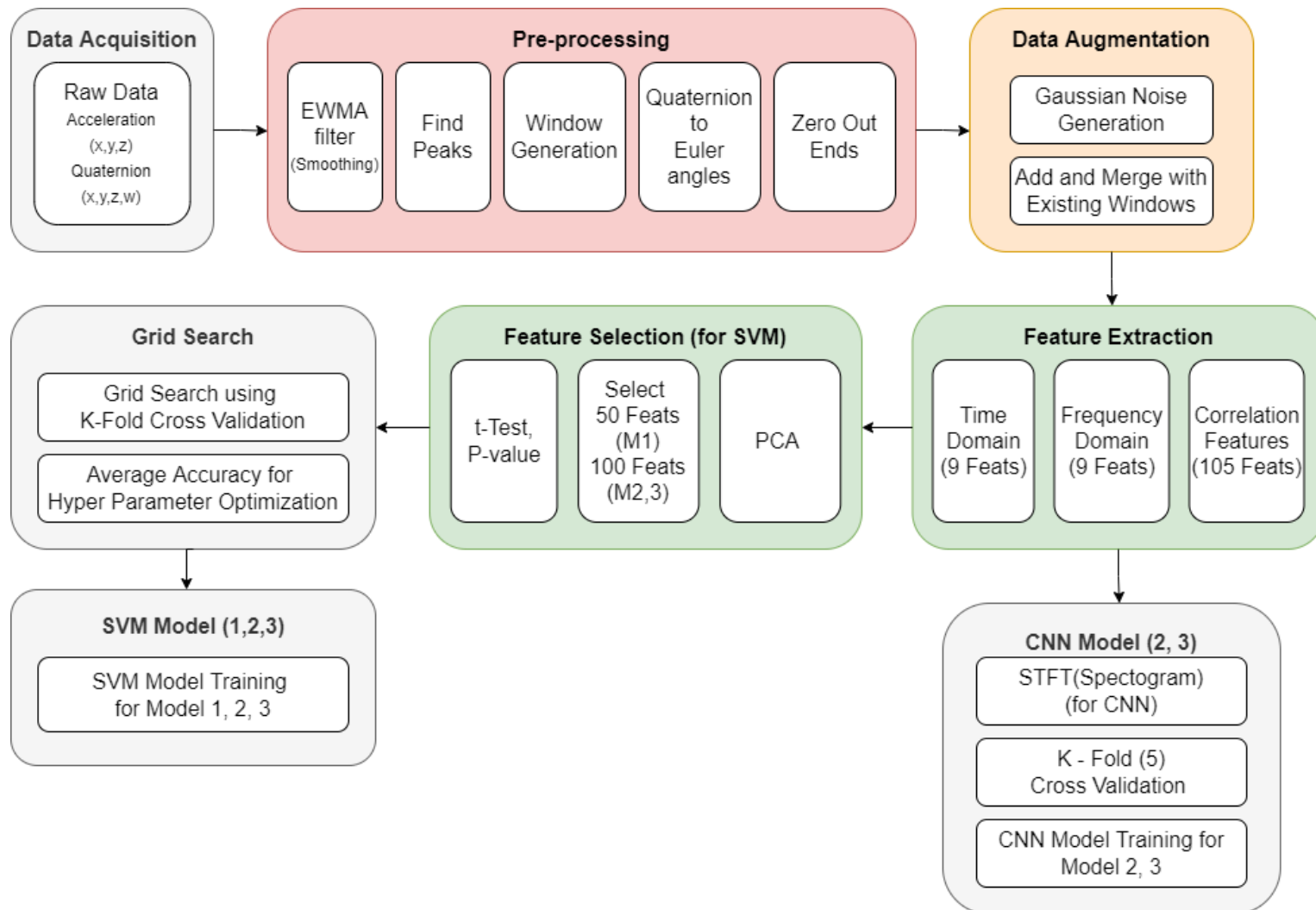
2주차

- 다중 센서 데이터 로깅 코드 작성
- 웨어러블 로봇 콘텐츠 Ideation (인식 동작 목록 정리)
- 데이터 수집 프로토콜 설정
- 센서 데이터 Pre-processing

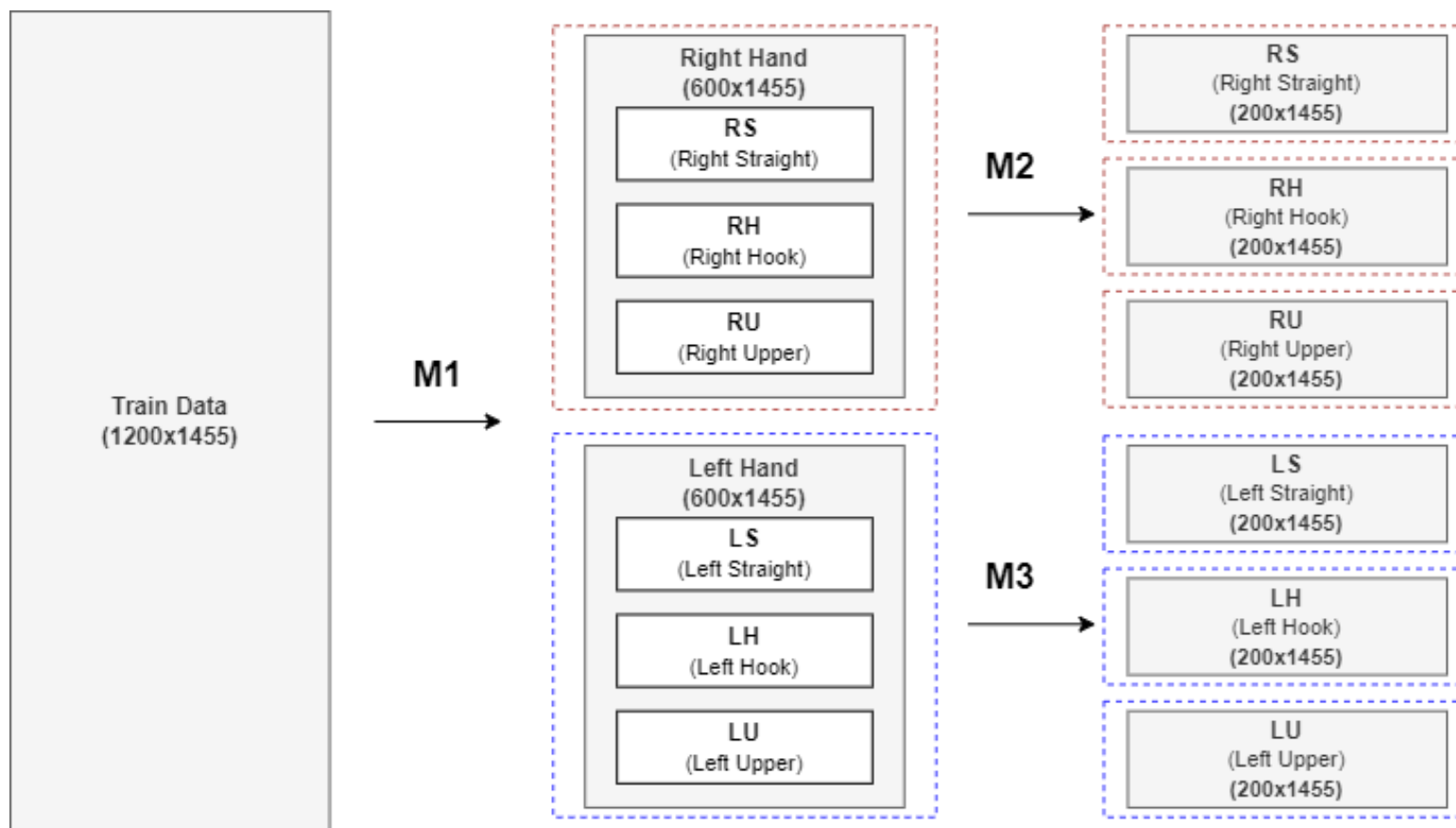
4주차

- 3진 분류 모델 구현 (one hand) (SVM, ANN)
- Recognition on-the-fly (one hand)
- 웨어러블 로봇 로그 데이터 수집

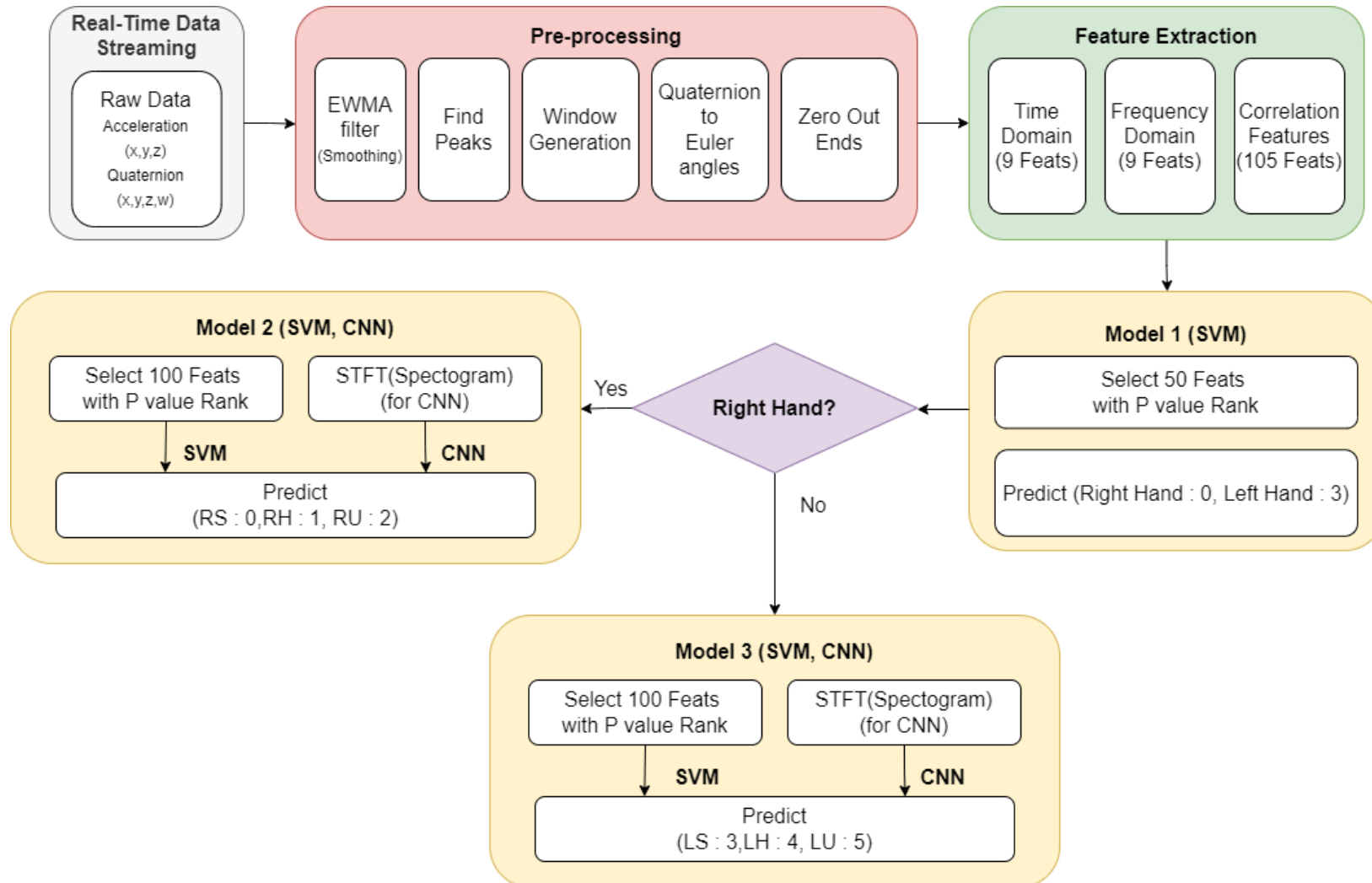
Project 전체 과정 – Classifier Model



Project 전체 과정 – Classifier Model 개요

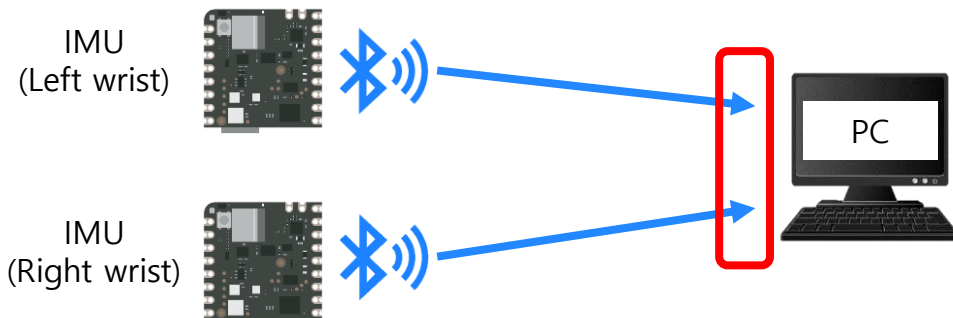
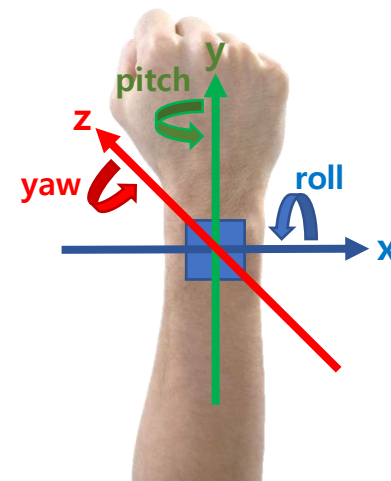


Project 전체 과정 – Recognition on-the-fly



▪ Data Acquisition – Nicla Sense ME – BHI260AP (6DoF, 무선)

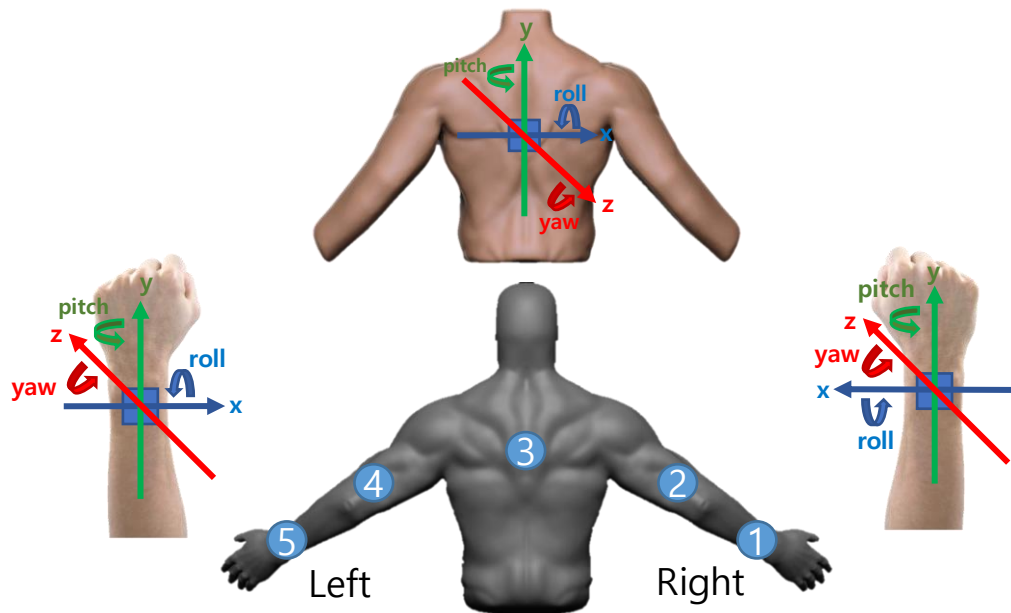
- 향후 웨어러블 로봇 제품에 적용될 예정인 무선 IMU 센서
- Python asyncio 라이브러리의 multi task를 이용한 Multiple BLE 통신
- 수집 Data : Acceleration (x,y,z), Gyroscope (x,y,z), Quaternion (x,y,z,w)
- Logging Frequency : 40Hz (One Hand), 12Hz (Two Hand)
- Trouble shooting:
 - 완전한 병렬식 코드 구동이 불가능. 센서 데이터가 완전히 교차로 수신되지 않음
 - 단순 flag 활용한 교차 수신 코드 작성
 - 교차 수신은 가능해졌지만, 로깅 frequency 저하 문제 발생



Project 진행 과정 – Classifier Model

▪ Data Acquisition(웨어러블 로봇) – EBIMU-9DOF (9DoF, 유선)

- 현재 웨어러블 로봇 제품에 적용되어 있는 유선 IMU 센서
- 등 부분에 있는 메인보드가 5개의 IMU로부터 센서 데이터 수신 후 메인보드와 컴퓨터간의 WiFi 통신으로 센서 메시지 수신. Unity Code 수정을 통해 Log Data 수집
- 수집 Data : Acceleration (x,y,z), Quaternion (x,y,z,w)
- Logging Frequency : 100Hz (Two Hand)



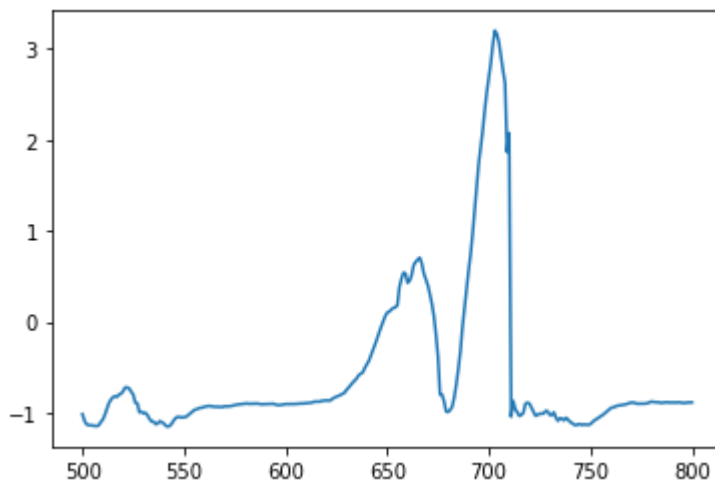
데이터 수집을 위한 IMU 센서 마운트

Project 진행 과정 – Classifier Model

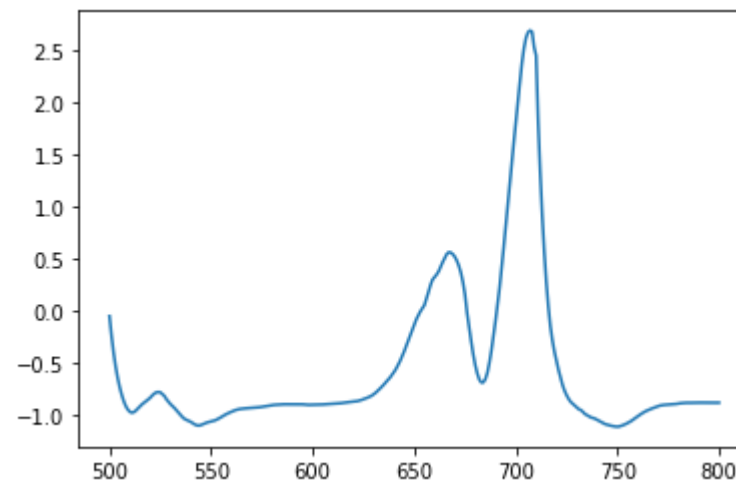
▪ Sensor Data Pre-processing – EWMA Filter

- EWMA (Exponential Weighted Moving Average) Filter
- 본 프로젝트의 데이터 수집에 사용된 웨어러블 로봇 제품은 성인 남성 체격에 맞추어 설계된 상태로, 특히 등, 팔에 부착된 2,3,4번 IMU의 경우 몸에 완벽히 고정되지 않아 진동이 심하고 잡음이 많은 상태.
- EWMA Filter를 적용하여 데이터를 Smoothing하고 잡음을 완화

$$EWMA(t) = \frac{Data_t + (1-\alpha)Data_{t-1} + (1-\alpha)^2 Data_{t-2} + \dots}{1 + (1-\alpha) + (1-\alpha)^2 + \dots} \quad \alpha = \frac{2}{N+1}$$



IMU(1번) y축 Acceleration
(EWMA Filter 적용 전)

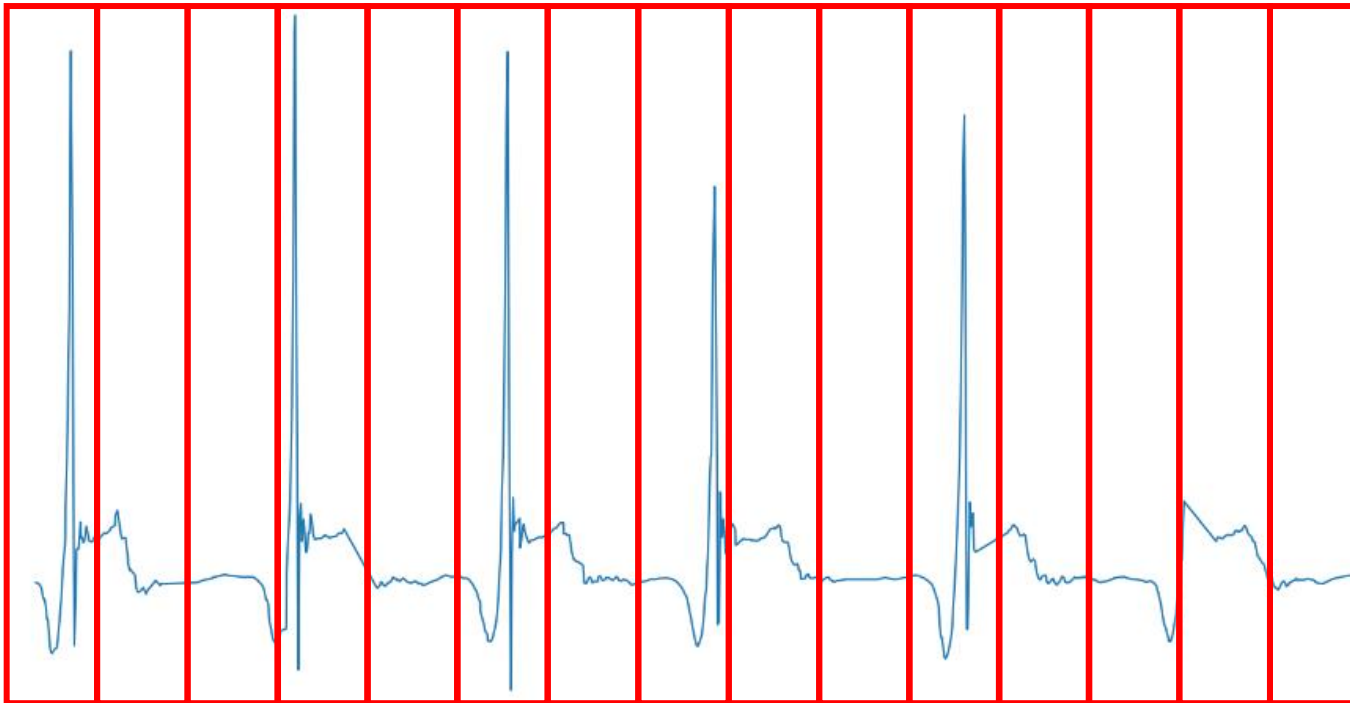


IMU(1번) y축 Acceleration
(EWMA Filter 적용 후)

Project 진행 과정 – Classifier Model

▪ Sensor Data Pre-processing – Windowing Technique

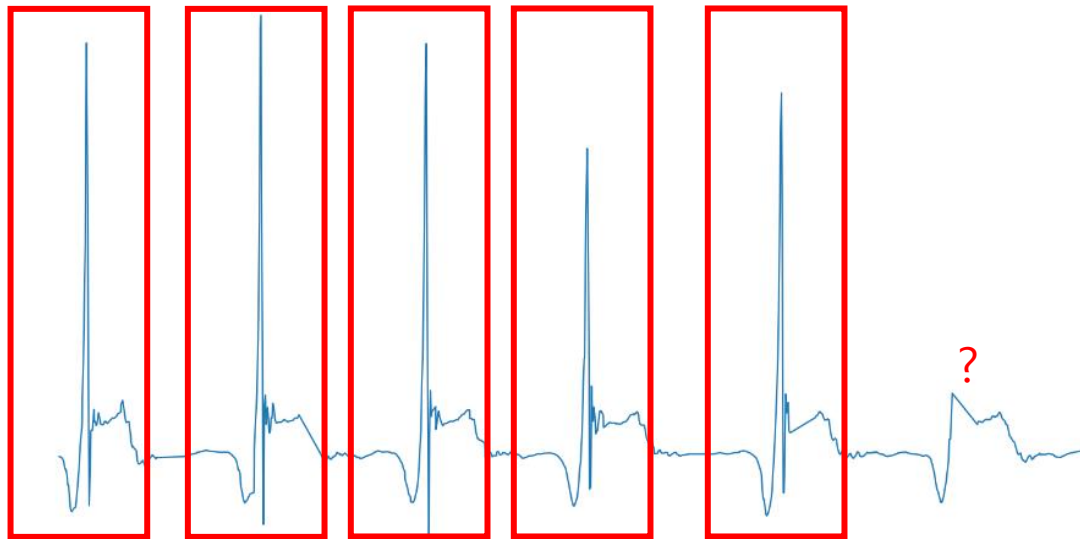
- 1. Fixed Sliding Window : 상태를 분류하는 것이 아닌 펀치 동작을 하나의 단위로 분류해야 하므로 적합하지 않다고 판단.



Project 진행 과정 – Classifier Model

▪ Sensor Data Pre-processing – Windowing Technique

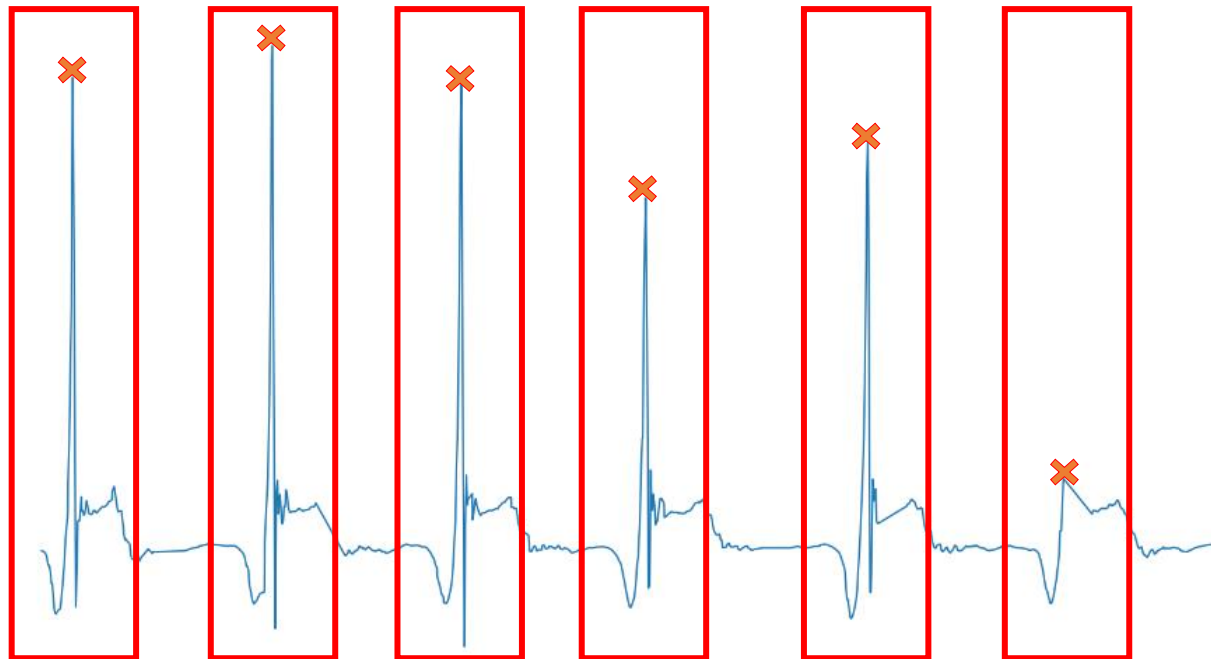
- 2. Behavior Definition Window : activity가 바뀐다는 것을 결정하기가 애매한 경우가 많아 적합하지 않다고 판단.
 - IMU 데이터로 부터 translation 정보를 도출하여 translation 정도가 특정 threshold 이하로 n초간 지속될 경우 동작이 끝났다고 판단
 - 가만히 서있는 rest 상태의 경우 끊임없이 window가 생성됨



Project 진행 과정 – Classifier Model

▪ Sensor Data Pre-processing – Windowing Technique

- 3. Event Definition Window : 특정 event를 기준으로 window 생성. 이 방식으로 결정
 - Y-axis의 acceleration 값의 peak 지점을 구한 뒤 ± 0.5 초 동안의 data를 하나의 window로 생성
 - 실제 logging data를 plotting 해보았을 때 peak지점이 명확하게 나타남을 확인 할 수 있어서 적합하다고 판단.



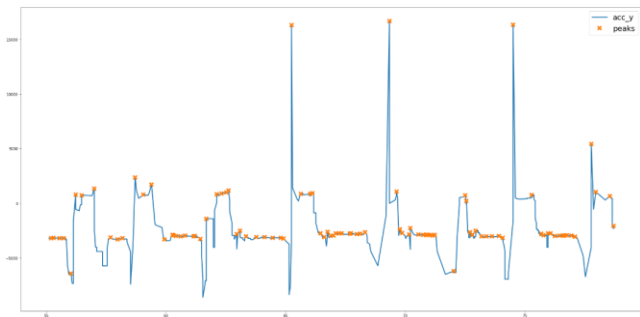
▪ Sensor Data Pre-processing – Find Peaks, Windowing (One Hand)

▪ 1. Training Data

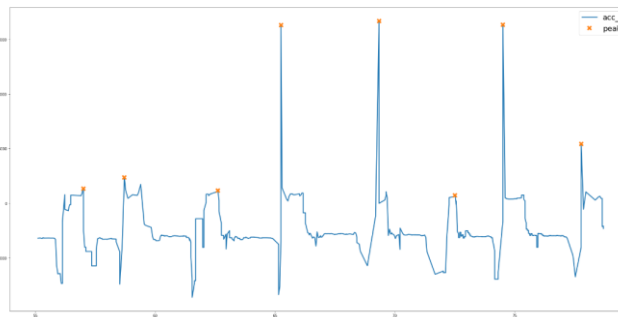
- Default 상태로 peak를 검출한 후 각 peak의 prominence를 계산하여 sort한 뒤 prominence가 높은 순서대로 peak 검출

▪ 2. Real-time Windowing Technique (Recognition on-the-fly)

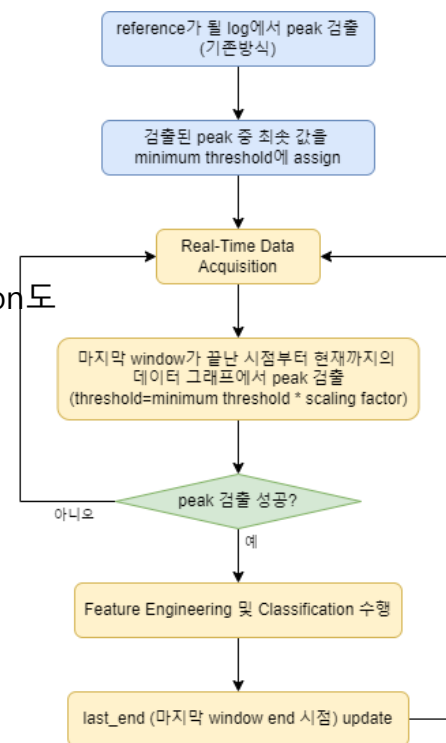
- 동일 인물의 log data를 받아서 기존의 방식으로 peak 검출 후 (peaks의 minimum 값 * Constant)를 minimum threshold로 설정하고 실시간으로 peak 검출
 - 실제 웨어러블 로봇에 적용 시에는 자세 Calibration 할 때 Punch Calibration도 함께 수행하는 방식으로 적용 가능



Peak 검출 (Default)



Peak 검출 (Prominence 고려)



Flow Chart (Real-time Peak 검출)

Project 진행 과정 – Classifier Model

▪ Sensor Data Pre-processing – Find Peaks, Windowing (Two Hand)

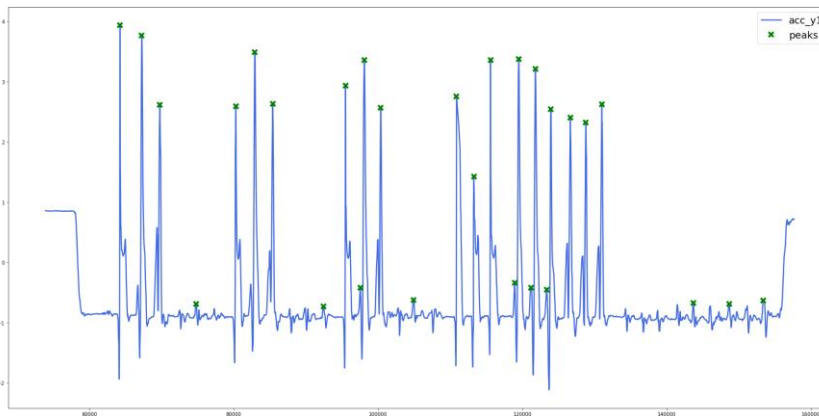
▪ 1. Training Data

- 'acc_y1' (오른손 손목 방향 가속도)과, 'acc_y5' (왼손 손목 방향 가속도) 기준으로 모두 peak를 검출한다. (검출 방식은 one hand때와 동일)
- Threshold를 이용하여 1차로 filtering
- 오른손, 왼손에서 검출한 peak 기준으로 생성한 window가 겹치는 경우에 대해 peak 값 크기가 큰 것, 그 다음은 prominence가 큰 것 순서로 우선 순위를 매겨 최종적으로 peak를 결정
- 최종적으로 결정한 peaks에 대해 좌우 50개 데이터 씩 총 100개의 fixed size window(약 1초)를 생성

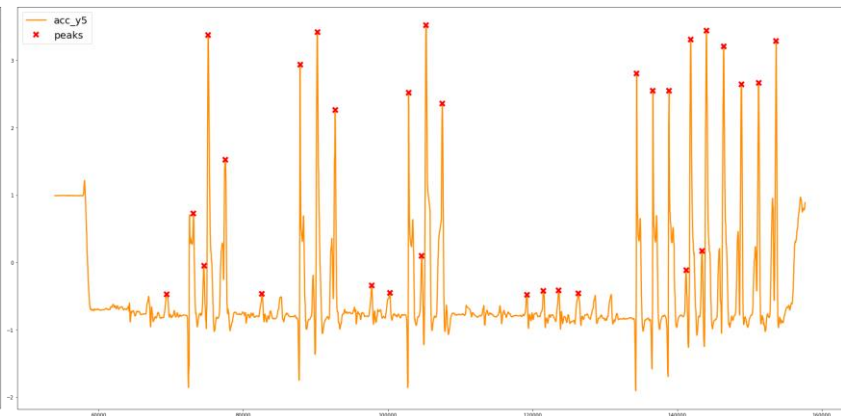
▪ 2. Real-time Windowing Technique (Recognition on-the-fly)

- 왼손, 오른손 펀치 동작들에 대한 로그 파일을 불러와서 peak 검출
- Peaks 중 minimum 값에 constant 값 (0.85)을 곱해 minimum threshold 설정
- Threshold를 반영하여 왼손, 오른손 모두로 부터 마지막 window의 end지점부터 현재의 데이터 까지 중 peak를 검출
- 왼손, 오른손 모두로 부터 peak가 검출될 경우, 해당 peak의 y축 acceleration값이 큰 쪽을 따름

▪ Sensor Data Pre-processing – Find Peaks, Windowing (Two Hand)



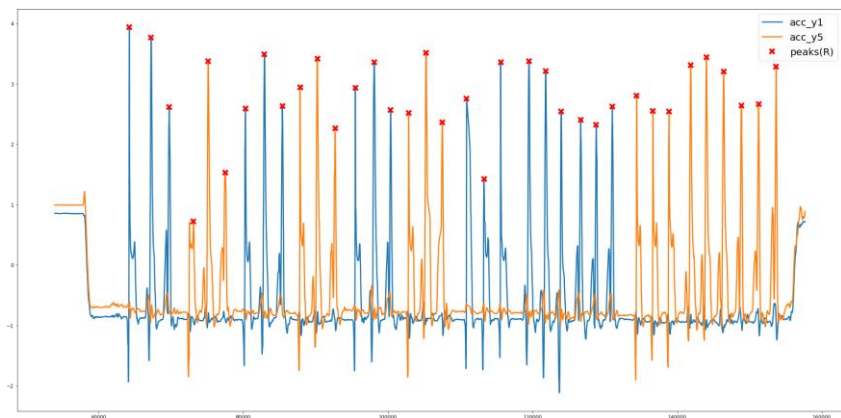
오른손 (1번 IMU) y축 가속도 값 peak 검출 결과



왼손 (1번 IMU) y축 가속도 값 peak 검출 결과

idx	acc	prominence
0	716.0	3.935111
1	979.0	3.764033
41	4306.0	3.517815
5	2324.0	3.493140
53	7563.0	3.441602
35	2979.0	3.419053
17	5439.0	3.374701
31	1661.0	3.372390
10	3674.0	3.359113
15	5100.0	3.356219
51	7359.0	3.310815
57	8373.0	3.283648
19	5642.0	3.214038
54	7743.0	3.203219
34	2769.0	2.939104
8	3433.0	2.935142
47	6743.0	2.806119
13	4754.0	2.752767
56	8158.0	2.666158
55	7961.0	2.642291
6	2544.0	2.631671

threshold filtering 후, acceleration 값,
prominence 순서로 peak들을 sort한 결과



최종 peak 검출 결과

▪ Sensor Data Pre-processing – Quaternion to Euler angles, Zero Out Ends

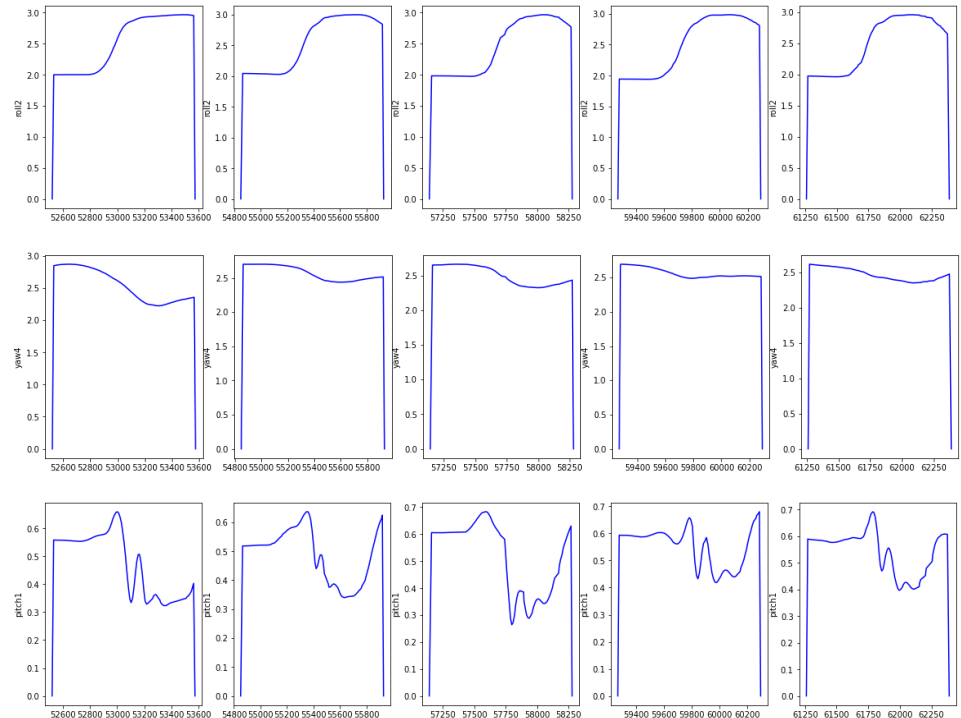
```
def quaternion_to_euler(windows):
    roll = [[]], pitch = [[]], yaw = [[]] # roll, pitch, yaw in radians
    for i in range(windows.shape[0]):
        for j in range(5):
            x=windows.loc[i,f"quat_x{j+1}"]
            y=windows.loc[i,f"quat_y{j+1}"]
            z=windows.loc[i,f"quat_z{j+1}"]
            w=windows.loc[i,f"quat_w{j+1}"]

            t0 = float(2*(w*x+y*z))
            t1 = float(1-2*(x*x+y*y))
            roll[j].append(np.arctan2(t0,t1))

            t2 = float(2*(w*y-z*x))
            t2 = 1 if t2>1 else t2
            t2 = -1 if t2<-1 else t2
            pitch[j].append(np.arcsin(t2))

            t3 = 2*(w*z+x*y)
            t4 = 1-2*(y*y+z*z)
            yaw[j].append(np.arctan2(t3, t4))

    for k in range(5):
        windows[f'roll{k+1}'] = roll[k]
        windows[f'pitch{k+1}'] = pitch[k]
        windows[f'yaw{k+1}'] = yaw[k]
    return windows
```



Project 진행 과정 – Classifier Model

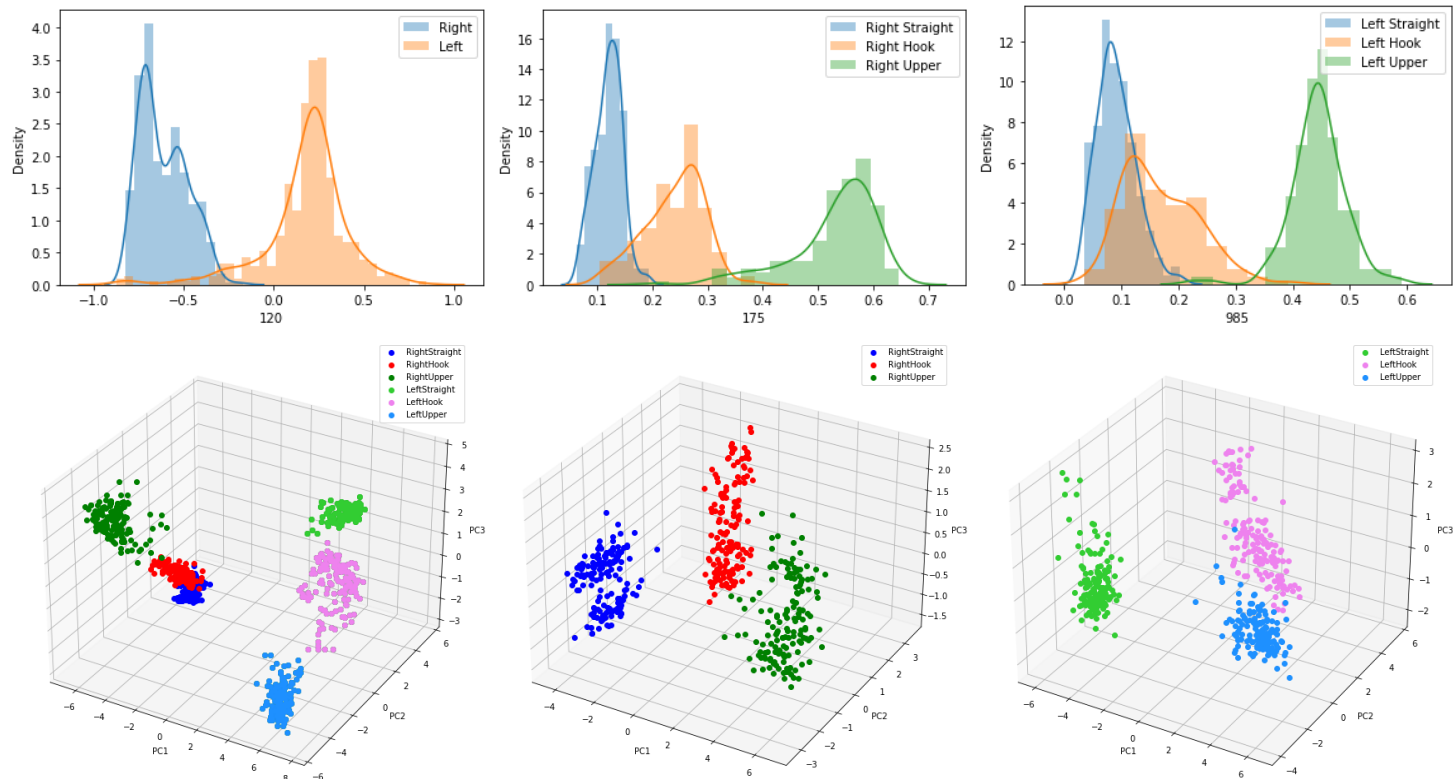
▪ Feature Extraction

- Time Domain Feature : 9 features * 6 sensor data * 5 sensors = 270
 - Maximum, minimum, mean, median, standard deviation, skewness, kurtosis, position of maximum, position of minimum
- Frequency Domain Feature : 9 features * 4 wavelet levels * 6 sensor data * 5 sensors = 1080
 - Maximum, minimum, mean, median, standard deviation, skewness, kurtosis, position of maximum, position of minimum
- Correlation Feature : 105 features (15C2)
 - Correlation between roll, pitch, yaw for 5 sensors

➤ **Total 1455 Features**

Feature Selection

- T-Test 수행 후 구분성 상위 50개(Model 1), 100개 (Model 2, 3) 선택

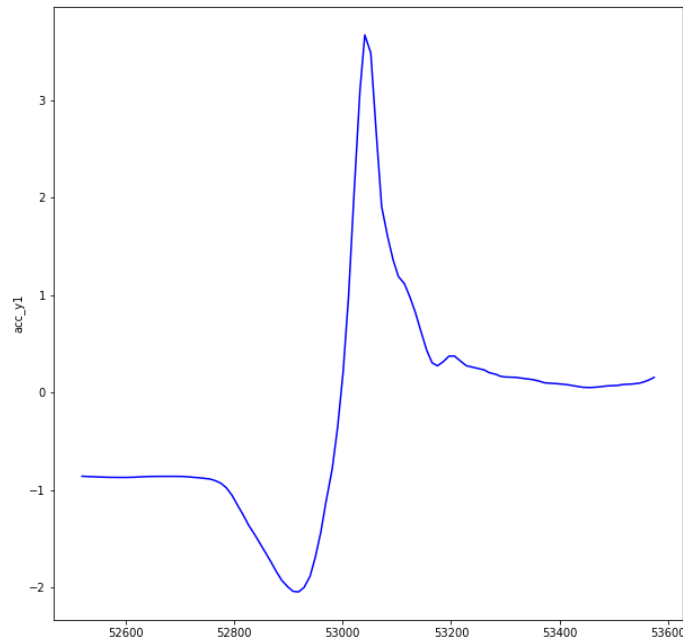


Model 1, 2, 3에 대한 구분성 상위 Rank PDF 그래프와 PCA 시각화(3D)

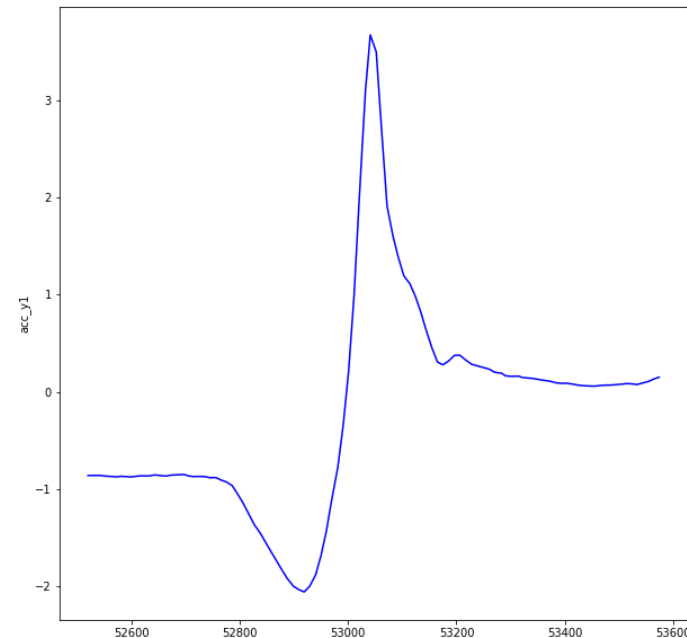
Project 진행 과정 – Classifier Model

▪ Sensor Data Pre-processing – Data Augmentation

- Data의 부족 문제에 대한 일부 해결을 위해 기존의 IMU Data에 Gaussian Noise를 추가하여 새롭게 window를 생성, 학습에 반영
 - Sensor Resolution을 standard deviation에 반영



Gaussian Noise 추가 전의 window



Gaussian Noise 추가하여 새롭게 생성한 window

Project 진행 과정 – Classifier Model

▪ Model 1 - SVM

- Right Hand, Left Hand 구분
- Test Data, Real-time 모두에서 높은 정확도 (100%)

	kernel	C	gamma	Accuracy
0	linear	0.01	0.01	1.000000
103	rbf	0.10	0.02	1.000000
93	rbf	0.02	0.10	1.000000
96	rbf	0.05	0.01	1.000000
97	rbf	0.05	0.02	1.000000
...	...	...	...	...
160	sigmoid	0.50	1.00	0.518889
154	sigmoid	0.20	1.00	0.518889
142	sigmoid	0.05	1.00	0.518889
166	sigmoid	1.00	1.00	0.518889
130	sigmoid	0.01	1.00	0.518889

Model 1 Grid Search 결과

```
Test_Score = svm_Model1.score(Test_Data_M1_Scaled, Test_Label_M1)
print('[Performance of SVM model (M1)] \n')
print('Test Score : {:.2f}%'.format(Test_Score*100))
```

[198] ✓ 0.5s Python

... [Performance of SVM model (M1)]

Test Score : 100.00%

Model 1 Test Accuracy

▪ Model 2,3 - SVM

- Hook, Upper Cut 동작에 대한 구분이 잘 되지 않음
- IMU로부터 position 값을 얻어 분류에 활용한다면 높은 분류 성능을 기대해볼 수 있을 것 같음

	kernel	C	gamma	degree	Accuracy
0	linear	0.01	0.01	3.0	1.00
180	poly	0.10	0.20	3.0	1.00
164	poly	0.05	0.50	5.0	1.00
165	poly	0.05	1.00	3.0	1.00
166	poly	0.05	1.00	4.0	1.00
...	...	...	...	...	...
394	sigmoid	0.10	0.50	4.0	0.56
395	sigmoid	0.10	0.50	5.0	0.56
351	sigmoid	0.02	0.50	3.0	0.56
353	sigmoid	0.02	0.50	5.0	0.56
352	sigmoid	0.02	0.50	4.0	0.56

420 rows × 5 columns

Model 2 Grid Search 결과

	kernel	C	gamma	degree	Accuracy
0	linear	0.001	0.01	3.0	1.00
227	rbf	0.001	0.50	5.0	1.00
253	rbf	0.020	0.01	4.0	1.00
252	rbf	0.020	0.01	3.0	1.00
248	rbf	0.010	0.50	5.0	1.00
...	...	...	...	...	...
397	sigmoid	0.050	1.00	4.0	0.56
396	sigmoid	0.050	1.00	3.0	0.56
417	sigmoid	0.100	1.00	3.0	0.56
418	sigmoid	0.100	1.00	4.0	0.56
419	sigmoid	0.100	1.00	5.0	0.56

420 rows × 5 columns

Model 3 Grid Search 결과

```

Test_Score = svm_Model2.score(Test_Data_M2_Scaled, Test_Label_M2)
print('Performance of SVM model (M2) \n')
print('Test Score : {:.2f}%'.format(Test_Score*100))

[202] ✓ 0.1s Python
... [Performance of SVM model (M2)]

Test Score : 100.00%

```

Model 2 Test Accuracy

```

Test_Score = svm_Model3.score(Test_Data_M3_Scaled, Test_Label_M3)
print('Performance of SVM model (M3) \n')
print('Test Score : {:.2f}%'.format(Test_Score*100))

[204] ✓ 0.1s Python
... [Performance of SVM model (M3)]

Test Score : 77.78%

```

Model 3 Test Accuracy

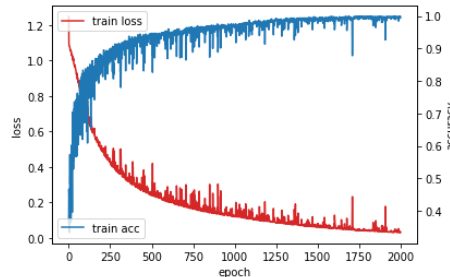
■ Model 2,3 – CNN

- Model 1의 경우 구분성이 높아 SVM 만으로 충분히 구분이 가능하므로, 구분성이 뛰어나지 않은 Model 2,3에 대해 CNN을 이용하여 분류 모델을 구현
- CNN 모델의 특성상 본 데이터와는 잘 맞지 않음

[Result of K-fold Cross Valid]

Fold 1: 94.44%
Fold 2: 95.56%
Fold 3: 98.89%
Fold 4: 93.33%
Fold 5: 87.78%
* Average accuracy : 94.00%

Model 2(CNN) Fold 검증



Model 2(CNN) Train Loss, Accuracy

```
Loss, Accuracy = cnn_Model2.evaluate(Test_Data_M2_CNN, M2_Test_Label_forCNN, verbose=0)
print('Performance of CNN model (M2) \n')
print('Accuracy : {:.2f}%'.format(Accuracy*100))
```

[206] ✓ 0.4s Python

... [Performance of CNN model (M2)]

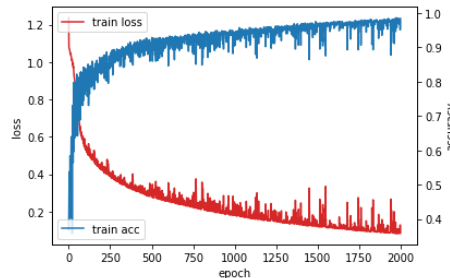
Accuracy : 61.11%

Model 2(CNN) Test Accuracy

[Result of K-fold Cross Valid]

Fold 1: 91.11%
Fold 2: 97.78%
Fold 3: 92.22%
Fold 4: 92.22%
Fold 5: 95.56%
* Average accuracy : 93.78%

Model 3(CNN) Fold 검증



Model 3(CNN) Train Loss, Accuracy

```
Loss, Accuracy = cnn_Model3.evaluate(Test_Data_M3_CNN, M3_Test_Label_forCNN, verbose=0)
print('Performance of CNN model (M3) \n')
print('Accuracy : {:.2f}%'.format(Accuracy*100))
```

[195] ✓ 0.2s Python

... [Performance of CNN model]

Accuracy : 77.78%

Model 3(CNN) Test Accuracy

Project 진행 과정 – Classifier Model

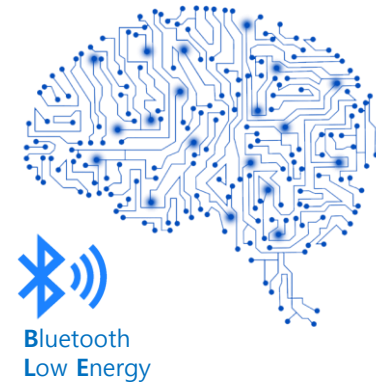
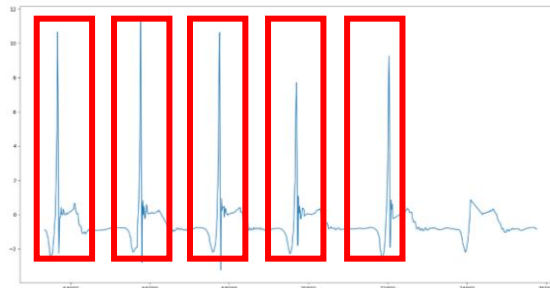
- Recognition on-the-fly

■ 배운 점

- Action Recognition
- Machine Learning, Deep Learning

- Windowing Technique

- BLE communication



Future Works

Data 부족 문제

- 다양한 신체 조건, 성별, 연령대의 피실험자로부터 다양한 데이터 얻기
- Bootstrapping, Boosting 등 여러가지 Data Augmentation 기법 이용

Windowing Technique

- 추후 웨어러블 로봇이 와이어 구동방식으로 개조 되면 엔코더를 통해 와이어가 풀린 정도를 파악 가능. 이 때 와이어가 특정 threshold 이상 늘어날 때 까지를 동작의 끝으로 인식하여 window 생성
- 실제 웨어러블 로봇에 적용된다면 동작이 끝날 때 (ex. 미트에 손이 닿았다고 판단될 때) haptic 이 발생되므로 이 외부의 event를 받아 window 생성이 가능할 것으로 예상.

IMU 데이터의 오차

- IMU가 첫 calibration 이후 시간이 흐름에 따라 drift 오차 누적
- Magnetometer 데이터를 활용한 Kalman Filter 구현으로 drift 해결

하드웨어 개선 방향

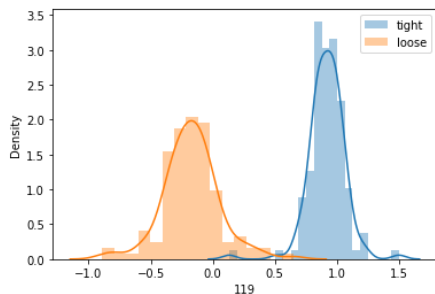
- 웨어러블 로봇 착용 후 팔 부분의 IMU (2,4번)가 계속 움직이는 현상
- 2,4번 IMU위치를 팔 윗 부분으로 변경하거나 아예 제거

분류 모델 개선

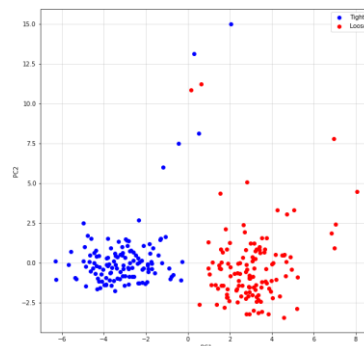
- SVM, ANN, CNN 외에도 더 다양한 모델을 적용해 보고 최적의 모델을 찾는 과정 필요
- IMU로부터 position estimation 과정을 수행하고 이 정보를 이용하여 특징을 새로 추출하여 모델에 적용 (Hook, Upper Cut 동작에 대한 구분성 보완)

하체용 웨어러블 로봇 log 수집 및 분석

- 브레인 스토밍에 앞서, GEMS LAB에서 현재 개발 진행 중인 하체용 웨어러블 로봇을 직접 착용 후, 특정 상황들에 대한 로그를 수집.
 - 1. WALK_RESIST 모드로 2분간 걷기 (허벅지 스트랩 꼭 조인 상태)
 - 2. WALK_RESIST 모드로 2분간 걷기 (허벅지 스트랩 약간 헐거운 상태 ① - 13cm)
 - 3. WALK_RESIST 모드로 2분간 걷기 (허벅지 스트랩 매우 헐거운 상태 ② - 11cm)
 - 4. WALK_RESIST 모드로 걷다가 도중에 스트랩 풀기
- 1번과 3번 케이스의 데이터를 fixed window로 분할하여 특성 추출 후 구분성을 간단히 확인해보았음.
 - 시계열 data를 50개씩 끊어 fixed window 생성
 - Maximum, minimum, mean, median, standard deviation, skewness, kurtosis, position of maximum, position of minimum의 9가지 feature를 time domain과 frequency domain으로 나누어 추출, 구분성 상위 30개 feature를 이용하여 확인해보았음
 - Future Work : 구분성이 높게 나타난 feature에 대해 탐구해보고 스트랩의 헐거움 정도를 파악할 수 있는 방안에 대해 브레인 스토밍 해보아야 함.



구분성 상위 Rank(1번째) PDF 그래프



PCA 시각화

```
Test_Score = SVM_model_Final.score(Test_Data_Scaled, Test_Label)
print('[Performance of SVM model] \n')
print('Test Score : {:.2f}%'.format(Test_Score*100))

[138] Python

... [Performance of SVM model]

Test Score : 98.15%
```

임시 test data에 대한 정확도

마지막 인사

Q&A

Thank you 😊